Handson Example Oracle 19c

Workshop 1

Part A: Writing Good SQL

Objectives:

- Learn about inequality conditions
- Learn about equality conditions
- Learn about `NULL` usage
- Learn about the effects of functions on index usage
- Learn to tune sorts for `ORDER BY` clauses
- Identify and tune implicit sort operations that are caused by `SELECT DISTINCT`
- Learn to tune `GROUP BY` operations and group functions
- Learn to tune set operators (`UNION`, `MINUS`, `INTERSECT`)

This workshop can be done entirely in the SQL*Plus environment. Try to work in small groups and discuss the workshop results. Each time you load a new SQL statement, try to predict what the optimizer will do before running the statement. Take notes during the workshop as an aid for the wrap-up discussion.

1. Open a terminal window. Change directory to the `workshop` directory. Log in to SQL*Plus as **`SH`** with password **`SH`**. Use the **`attox.sql`** script to enable `AUTOTRACE TRACEONLY EXPLAIN` to suppress statement output and produce execution plans; check the settings with `SHOW AUTOTRACE`.

```
$cd workshop
$ sqlplus sh/sh
SQL> show autotrace
SQL> @attox
SQL> show autotrace
```

Student Observations:

2. Get and run queries **`ws_01_01.sql`**, **`ws_01_01a.sql`**, **`ws_01_02.sql`**, **`ws_01_02b.sql`**, and **`ws_01_03.sql`**. First remove all indexes from the `CUSTOMERS` table by running the **`dai.sql`** script.

Note: `dai.sql` removes all non-primary-key indexes.

Note: The `CUST_ID` column is the primary key and is still indexed.

```
SQL> @dai
on which table: customers

SQL> get ws_01_01
  1  SELECT cust_first_name
  2 , cust_last_name
  3 FROM customers
  4* WHERE cust_id = 1030
```

```
SQL>@ws_01_01
SQL> get ws_01_01a
  1 SELECT cust_first_name
  2 , cust_last_name
  3 FROM customers
  4* WHERE cust_id <> 1030
SQL>@ws_01_01a
SQL> get ws_01_02
  1 SELECT cust_first_name
  2 , Cust_Last_Name
  3 FROM customers
  4* WHERE cust_id < 10
SQL>@ws_01_02
SQL> get ws_01_02b
  1 SELECT cust_first_name
  2 , cust_last_name
  3 FROM customers
  4* WHERE cust_id < 10000
SQL>@ws_01_02b
SQL> get ws_01_03
  1 SELECT cust_first_name
  2 , cust_last_name
  3 FROM customers
  4* WHERE cust_id between 70 AND 80
SQL>@ws_01_03
```

Analyze the results of these queries and determine when the Oracle optimizer can use indexes.

Indexes may be used for three types of conditions:

- Equality search (`ws_01_01.sql`)
- Unbounded range (`ws_01_02` and `ws_01_02b.sql`)
- Bounded range (`ws_01_03.sql`)

However, even then the optimizer considers the selectivity of the operation before using an index. (If you have time, try changing the values in **`ws_01_03.sql`** to `70000` and `80000` and see what happens.) The index is not used if the `NOT EQUAL` (<>) operator is present.

Note: Index usage may be forced using an `INDEX` hint.

Student Observations:

3. Now create an index on the `CUST_CREDIT_LIMIT` column of the `CUSTOMERS` table by using the **`ci.sql`** script. Explain the queries in **`ws_01_04.sql`** and **`ws_01_05.sql`**. Use the **`rp.sql`** script to retrieve the information from `PLAN_TABLE` by using the `dbms_xplan` package. Make sure that you have disabled `AUTOTRACE` by running **`atoff.sql`**.

```
SQL> @atoff
SQL> @ci
on which table : customers
on which column(s): cust_credit_limit
Creating index on: customers cust_credit_limit
```

```
SQL> get ws_01_04
  1 explain plan for
  2  SELECT cust_id
  3 FROM customers
  4 WHERE cust_credit_limit*1.10 = 11000
SQL>@ws_01_04
SQL> @rp
SQL> get ws_01_05
  1 explain plan for
  2 SELECT cust_id
  3 FROM customers
  4 WHERE cust_credit_limit = 30000/2
SQL>@ws_01_05
SQL> @rp
```

The results show that although the `CUST_CREDIT_LIMIT` column is indexed, the index is not used by default. This can happen if the indexed column is part of an expression in the `WHERE` clause. An index is usable only if the indexed column appears clean in the `WHERE` clause and even then may only be used based on selectivity. The `CUST_CREDIT_LIMIT` column has poor selectivity because it has only eight values, which may result in the index being ignored at times.

Notice how the optimizer automatically filters on `15000`.

Student Observations:

4. Create an index on the `CUST_LAST_NAME` column of the `CUSTOMERS` table. View the explain plan for queries **`ws_01_06.sql`**, **`ws_01_07.sql`**, **`ws_01_07a.sql`**, **`ws_01_07b.sql`**, and **`ws_01_07c.sql`** by using **`rp.sql`**.

```
  1 explain plan for
  2 SELECT cust_id
  3 FROM customers
  4 WHERE upper(cust_last_name) ='KING'
SQL>@ws_01_07a
```

```
SQL> @rp
SQL> get ws_01_07b
  1 explain plan for
  2 SELECT cust_id
  3 FROM customers
  4 WHERE cust_last_name ='King'
SQL>@ws_01_07b
SQL> @rp
SQL> get ws_01_07c
  1 explain plan for
  2 SELECT cust_id
  3 FROM customers
  4 WHERE cust_last_name = initcap('KING')
SQL>@ws_01_07c
```

You see that how you write your SQL has a significant impact on whether indexes are used or not. The indexed column should be clean. As soon as you apply any function, index usage does not happen by default. Also an index may not be used for selectivity reasons. You may, however, force index usage with hints.

Student Observations:

5. Now start query **`ws_01_08.sql`** and compare the execution plan with **`ws_01_07.sql`**. How do you explain the difference?

```
SQL> get ws_01_08
  1  explain plan for
  2  SELECT cust_last_name
  3  FROM customers
  4  WHERE cust_last_name like 'S%'
SQL>@ws_01_08
SQL> @rp
```

Student Observations:

6. Create the explain plan for the query in **`ws_01_09.sql`**. Then use the **`rp.sql`** script to retrieve the information in a meaningful way from the `PLAN_TABLE`. Try to explain why the index is not used: The indexed column is clean, and the search pattern does not start with a wildcard.

Hint: Look at the filter.

```
SQL> get ws_01_09
  1 explain plan for
  2 SELECT cust_last_name
  3 FROM customers
  4 WHERE cust_id like '7%'
SQL>@ws_01_09
SQL>@rp
```

Student Observations:

7. In this exercise, you investigate the treatment of NULL-values and use the CUSTOMERS table for that purpose. First, run the **ws_01_10a.sql** script to remove some values from the CUST_EMAIL column. Second, create an index on the CUST_EMAIL column of the CUSTOMERS table; this column contains many NULL-values. Then start query **ws_01_10b.sql**.

```
SQL> @atoff
SQL> @ws_01_10a
SQL> describe customers
SQL> @ci
on which table : customers
on which column(s): cust_email
SQL> get ws_01_10b
  1  explain plan for
  2  SELECT cust_email
  3 FROM customers
  4* WHERE cust_email is null
SQL>@ws_01_10b
SQL>@rp
```

Student Observations:

To explain why the index is not used in this case, you should be aware that the Oracle Server does not store any references to NULL-values in a regular B*-tree index. That is why the only way to find rows containing NULL values is to perform a full-table scan. In the case of a concatenated index, if all columns in the index are NULL, then the same explanation applies.

Student Observations:

8. Now suppose that you are interested in all rows that do *not* contain a NULL value. Start **ws_01_11.sql** and compare with **ws_01_10b.sql**.

```
SQL> get ws_01_11
  1 explain plan for
SELECT cust_id
FROM customers
  4* WHERE cust_email is not null
SQL>@ws_01_11
SQL> @rp
```

The index is again not used—but for a different reason: The optimizer decision is based on selectivity considerations. You can assist the optimizer in choosing a fast full index scan by creating a concatenated index on the CUST_ID and CUST_EMAIL columns.

Part B: Sorting, Grouping, and Set Operators

In this workshop, you concentrate on tuning explicit and implicit sort operations. Sorting is a common operation. If the Oracle Server is able to perform all sort activity in memory, then the performance is probably acceptable. However, sometimes the Oracle Server writes intermediate results to disk. Therefore, SQL*Plus `AUTOTRACE` shows two statistics: sorts (memory) and sorts (disk). Sometimes you can avoid sort operations by creating indexes, or you can suppress implicit sorts that are not needed for the result that you want.

1. Drop all indexes on the `CUSTOMERS` table by using the **`dai.sql`** script. Note that the index that is related to the primary key constraint cannot (and does not need to) be dropped. View and execute query **`ws_01_13.sql`**:

```
SQL> @dai on which table: customers
SQL> get ws_01_13.sql
  1  explain plan for SELECT cust_first_name
  2  , cust_last_name
  3  , cust_credit_limit
  4  FROM customers
  5* ORDER BY cust_credit_limit
SQL>@ws_01_13
SQL>@rp
```

Student Observations:

The Oracle Server must perform a sort operation.

2. Sorting can be avoided by creating appropriate indexes, so investigate what happens when you create an index on the `CUST_CREDIT_LIMIT` column:

```
SQL> @ci
on which table : customers
on which column(s): cust_credit_limit
Creating index on: customers cust_credit_limit
SQL> @ws_01_13
SQL> @rp
```

The results show that although the `CUST_CREDIT_LIMIT` column is indexed, the optimizer does not use the index (to avoid a sort operation).

Student Observations:

3. Get and run **ws_01_14.sql** and compare the results with **ws_01_13.sql**.

```
SQL> get ws_01_14
  1  explain plan for SELECT cust_first_name
  2 , cust_last_name
  3 , cust_credit_limit
  4 FROM  customers
  5* ORDER BY cust_id
SQL>@ws_01_14
SQL> @rp
```

Is the index on CUST_ID used?

Student Observations:

4. Get and run ws**_01_15.sql**. Does the WHERE clause make a difference?

```
SQL> get ws_01_15
  1  explain plan for SELECT cust_first_name
  2  , cust_last_name
  3  , cust_city
  4  FROM customers
  5  WHERE cust_city = 'Paris'
  6* ORDER BY cust_id
SQL>@ws_01_15
SQL> @rp
```

Student Observations:

5. Investigate what happens if you create an additional index on the CUST_CITY column:

```
SQL> @ci
on which table : customers
on which column(s): cust_city

SQL> @li
List indexes on table: customers
SQL> @ ws_01_15
SQL> @rp
```

Which index is used?

Student Observations:

6. Now get and run queries **ws_01_16.sql**, **ws_01_17.sql**, and **ws_01_18.sql**.

```
SQL> get ws_01_16
  1 explain plan for SELECT max(cust_credit_limit)
  2* FROM customers
SQL>@ws_01_16
SQL> @rp
SQL> get ws_01_17
  1 explain plan for SELECT max(cust_credit_limit+1000)
  2* FROM customers
SQL>@ws_01_17
SQL> @rp
SQL> get ws_01_18
  1 explain plan for SELECT max(cust_credit_limit*2)
  2* FROM customers
SQL>@ws_01_18
SQL> @rp
```

This shows that an index can be useful to retrieve a maximum value (and a minimum value). If no index is available, the optimizer must scan the full table and perform a sort. Sometimes an operation on the indexed column value (such as in **ws_01_18.sql**) prevents the index from being used. If you have time, try creating an index on `(CUST_CREDIT_LIMIT*2)`. Does this help?

Student Observations:

7. Start query **ws_01_19.sql.** A `WHERE` clause is added and you see the result.

```
SQL> get ws_01_19
  1  explain plan for SELECT max(cust_credit_limit)
  2  FROM customers
  3* WHERE cust_city = 'Paris'
SQL>@ws_01_19
SQL> @rp
```

Student Observations:

8. Start query **ws_01_20.sql**. This query shows that to evaluate a `SELECT DISTINCT`, the index is used.

```
SQL> get ws_01_20
  1 explain plan for SELECT distinct cust_city
  2* FROM customers
SQL>@ws_01_20
SQL> @rp
```

Student Observations:

Drop the index on the CUST_CITY column and run **ws_01_19.sql** again.

```
SQL> @li
List indexes on table: customers
SQL> drop index cust_cust_city_idx;
SQL> @ws_01_19
SQL> @rp
```

Student Observations:

The optimizer chooses a full-table scan in the absence of the index. Now run **ws_01_20.sql** and obtain the explain plan

```
SQL> @ws_01_20
SQL> @rp
```

Student Observations:

9. The following queries investigate the SQL set operators. These operators unconditionally result in sort operations, regardless of the presence of indexes. To investigate this, create any indexes you like. The sorts are needed because the SQL set operators are supposed to filter duplicate rows from the result.

```
SQL> get ws_01_21
  1  explain plan for
  2  SELECT cust_last_name FROM customers
  3  Where cust_city = 'Paris'
  4  intersect
  5  SELECT cust_last_name FROM customers
  6* Where cust_credit_limit < 10000
SQL>@ws_01_21
SQL> @rp

SQL> get ws_01_22
  1  explain plan for
  2  SELECT cust_last_name FROM customers
  3  Where cust_city = 'Paris'
  4  minus
  5  SELECT cust_last_name FROM customers
  6* Where cust_credit_limit > 10000
```

```
SQL>@ws_01_22
SQL> @rp
SQL> get ws_01_23
  1  explain plan for
  2  SELECT cust_last_name FROM customers
  3  Where cust_city = 'Paris'
  4  UNION
  5  SELECT cust_last_name FROM customers
  6* Where cust_credit_limit < 10000
SQL>@ws_01_23
SQL> @rp
```

Student Observations:

There is one exception: The `UNION ALL` operator does not perform a sort and does not filter duplicate rows. Use the `UNION ALL` operator if you are sure that there are no duplicate rows or that duplicate rows cause no semantic problems.

```
SQL> get ws_01_24
  1  explain plan for
  2  SELECT cust_last_name FROM customers
  3  Where cust_city = 'Paris'
  4  UNION ALL
  5  SELECT cust_last_name FROM customers
  6* Where cust_credit_limit < 10000
SQL>@ws_01_24
SQL> @rp
```

Student Observations:

10. Examine the `GROUP BY` operator by using the script **`ws_01_25.sql`**. Try indexing the `GROUP BY` column. What happens?

```
SQL> get ws_01_25
  1  explain plan for SELECT cust_city
  2 , avg(cust_credit_limit)
  3 FROM customers
  4* GROUP BY cust_city
SQL>@ws_01_25
SQL> @rp
SQL> @ci
on which table : customers
on which column(s): cust_city
Creating index cust_cust_city_idx
SQL> @ws_01_25
SQL> @rp
```

Student Observations:

Now examine the following two SQL statements, which are logically equivalent:

```
SQL> get ws_01_26
  1  explain plan for SELECT cust_city
  2  , avg(cust_credit_limit)
  3  FROM customers
  4  WHERE cust_city = 'Paris'
  5* GROUP BY cust_city
SQL>@ws_01_26
SQL> @rp
Student Observations:
SQL> get ws_01_27
  1 explain plan for
  2 SELECT cust_city
  3  , avg(cust_credit_limit)
  4  FROM customers
  5  GROUP BY cust_city
  6* having cust_city = 'Paris'
SQL>@ws_01_27
SQL> @rp
```

The index on the `CUST_CITY` column may be used in **`ws_01_26.sql`** to reduce the set of rows that must be sorted. This is not possible in **`ws_01_27.sql`** because the `HAVING` clause is always evaluated after the `GROUP BY` clause. Note that **`ws_01_27.sql`** is a badly formulated SQL statement. A `HAVING` clause usually contains a group function: `COUNT`, `SUM`, `AVG`, `MIN`, or `MAX`.

Student Observations:

Part C: Subqueries

11. Log in as **HR/HR**. Set AUTOTRACE to TRACE ONLY.

```
SQL> @atto
```

12. View and execute the SQL statement in **ws_01_28.sql.**

```
SQL> get ws_01_28
  1  SELECT employee_id, last_name
  2  FROM   employees e1
  3  WHERE  e1.salary >
  4         (SELECT avg(salary)
  5          FROM employees e2
  6          WHERE e1.department_id = e2.department_id
  7*         GROUP BY e2.department_id)
SQL>@ws_01_28
```

This correlated subquery example processes all the rows of the outer table (C1). Each row in the outer table is checked against every row from the inner condition.

Student Observations:

If you reformulate the query to use a join between two tables, one of which is built up on the spot (an in-line view), does performance improve? Use the **ws_01_29.sql** script for this purpose.

```
SQL> get ws_01_29
  1  SELECT e1.employee_id, e1.last_name
  2  FROM   employees e1
  3  ,      (SELECT avg(salary) avg_sal, department_id
  4          FROM   employees
  5          GROUP BY department_id) a
  6  WHERE  e1.salary > a.avg_sal
  7* AND    e1.department_id = a.department_id
SQL> @ws_01_29
```

Student Observations:

Workshop Summary

After completing this workshop, you should have learned the following:

- Indexes can be used to avoid sorting. However, the optimizer must not miss any rows, which is why the index must be on a `NOT NULL` column.
- If any additional indexes are available to reduce the set of rows to be sorted, those paths may be more effective. Sorts on small sets of rows usually perform well, and this approach reduces throwaway (retrieving rows that are not needed for the result set).
- Maximum and minimum values can be retrieved from indexes, with certain restrictions (for example, as long as there is no `WHERE` clause and no `GROUP BY` clause).
- `SELECT DISTINCT`, `GROUP BY`, `UNION`, `MINUS`, and `INTERSECT` all result in an unconditional sort operation. This sort operation cannot be suppressed. However, it can be tuned by trying to reduce the set of rows to be sorted.
- Try to use the `WHERE` clause for row-level predicates instead of the `HAVING` clause.